Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Факультет программной инженерии и компьютерной техники

Отчёт по лабораторной работе №1

по дисциплине «Системы искусственного интеллекта»

Модуль 1. Базы знаний и онтологии

Тема: «Видеоигры: свойства, жанры и разработчики»

Выполнил:
студент группы P3319
Быков Лев

Преподаватель:
Болдырева Елена Александровна

г. Санкт-Петербург
2026

Содержание

1. Введение
2. Анализ требований
3. Изучение основных концепций и инструментов
4. Реализация системы искусственного интеллекта
    4.1. База знаний на Prolog
    4.2. Онтология в Protege
    4.3. Тестирование и отладка
5. Оценка и интерпретация результатов
6. Заключение
Приложение А. Структура проекта

1. Введение

Цель работы — освоить два способа представления знаний: логическое программирование на языке Prolog и онтологическое моделирование в среде Protege. Для этого на одной предметной области (видеоигры) построены база знаний на Prolog и эквивалентная ей онтология OWL 2, и на обеих реализованы запросы различной сложности.

Разработанная система решает прикладную задачу поддержки выбора видеоигры. Пользователь может узнать, какие игры подходят под набор условий: одиночный или многопользовательский режим, сюжетность, независимая разработка, жанр, студия. Отдельные правила выводят новые знания, которых нет в исходных фактах: например, рекомендацию игры для начинающего игрока или пары игр, созданных одной студией.

Значимость работы. Базы знаний и онтологии лежат в основе экспертных систем, рекомендательных сервисов, семантического веба и графов знаний. Prolog показывает, как знания описываются фактами и правилами, а ответы получаются автоматическим логическим выводом. Онтологии решают ту же задачу стандартизированным способом (W3C OWL), удобным для обмена знаниями между системами и проверки их непротиворечивости автоматическими reasoner'ами. Сравнение двух подходов на одних данных показывает их сильные стороны и принципиальные различия — прежде всего различие между предположениями о замкнутом и открытом мире.

Задачи работы:

1. разработать базу знаний на Prolog: не менее 20 унарных фактов, 10–15 бинарных фактов и 5–7 правил;
2. составить запросы разной сложности, включая логические «и», «или», «не», запросы с переменными и запросы, использующие правила;
3. перевести базу знаний в онтологию в Protege: классы, свойства, иерархию, индивиды и правила;
4. протестировать обе реализации и сравнить результаты.

2. Анализ требований

2.1. Требования к системе поддержки принятия решений

Система должна помогать пользователю выбрать видеоигру по его предпочтениям. Из этого вытекают функциональные требования:

- **Поиск по свойствам.** Найти все игры с заданным свойством (одиночные, многопользовательские, сюжетные, независимые и т.д.).
- **Комбинирование условий.** Поддержать конъюнкцию («сюжетная И одиночная»), дизъюнкцию («спокойная ИЛИ головоломка») и отрицание («без мультиплеера»).
- **Поиск по отношениям.** Найти игры заданной студии или жанра и, наоборот, студию по игре.
- **Вывод новых знаний.** Формулировать рекомендации по правилам, а не только возвращать сохранённые факты.
- **Проверяемость.** Результаты должны быть воспроизводимы и подтверждаться автоматическими тестами.

Нефункциональные требования: понятность кода и наличие комментариев, возможность расширения базы новыми играми без изменения правил, корректность логического вывода.

2.2. Требования к базе знаний

Требование задания	Реализация
Не менее 20 фактов с одним аргументом	20 фактов `game/1` и ещё 53 унарных факта-свойства: `singleplayer/1` (16), `multiplayer/1` (11), `indie/1` (9), `story_driven/1` (11), `competitive/1` (4), `cozy/1` (2)
10–15 фактов с двумя аргументами	15 фактов: `has_genre/2` (7), `developed_by/2` (8)
5–7 правил	7 правил
Запросы с «и», «или», «не»	Операторы `,` `;` `\+` в запросах и правилах
Комментарии	Каждый предикат и правило описаны комментарием

2.3. Требования к онтологии

- Каждому предикату Prolog должен соответствовать элемент онтологии: класс, объектное свойство или индивид.
- Должна быть построена иерархия классов.

- Правила Prolog должны быть выражены средствами OWL так, чтобы reasoner выводил те же результаты, что и Prolog.
- Онтология должна быть непротиворечивой (consistent).
- Должна быть продемонстрирована функциональность: визуализация и выполнение запросов (DL Query, SPARQL).

3. Изучение основных концепций и инструментов

3.1. Базы знаний и онтологии

База знаний — совокупность формально записанных сведений о предметной области и правил, по которым из этих сведений выводятся новые. В отличие от обычной базы данных, база знаний содержит не только данные, но и логику: ответ на запрос может не храниться явно, а быть выведен.

Онтология — формальное явное описание понятий предметной области и отношений между ними. Её основные элементы:

- **классы** (концепты) — категории объектов: «Игра», «Жанр», «Студия»;
- **индивиды** (экземпляры) — конкретные объекты: Portal 2, Valve;
- **объектные свойства** — связи между индивидами: «разработана студией»;
- **свойства-данные** — связи индивида со значением (число, строка);
- **аксиомы** — утверждения о классах и свойствах: подкласс, эквивалентность, непересекаемость, обратность, функциональность.

Онтологии на языке **OWL 2** основаны на **дескрипционных логиках** (Description Logics) — разрешимых фрагментах логики первого порядка. Благодаря этому **reasoner** (машина вывода, например HermiT) может автоматически классифицировать классы, определять принадлежность индивидов классам и проверять непротиворечивость онтологии.

Ключевое различие двух подходов — **предположение о мире**:

- **Замкнутый мир (CWA)**, Prolog: всё, что нельзя доказать, считается ложным (отрицание как неудача).
- **Открытый мир (OWA)**, OWL: отсутствие утверждения означает «неизвестно», а не «ложно».

Кроме того, в Prolog действует **предположение об уникальности имён** (разные атомы — разные объекты), а в OWL два разных имени могут обозначать один объект, если не заявлено обратное.

3.2. Prolog

Prolog — декларативный язык логического программирования. Программа на Prolog описывает *что* известно, а не *как* вычислять ответ. Основные конструкции:

- **факт** — безусловное утверждение: `game(portal_2).`;
- **правило** — условное утверждение «голова истинна, если истинно тело»: `social_game(G) :- game(G), multiplayer(G).`;
- **запрос** — вопрос к базе: `?- developed_by(G, valve).`

Ответ на запрос ищется механизмом **SLD-резолюции**: интерпретатор сопоставляет цель с фактами и головами правил с помощью **унификации**, перебирает варианты в порядке записи и при неудаче выполняет **откат** (backtracking) к последней точке выбора. Логические связки: запятая `,` — «и», точка с запятой `;` — «или», `\+` — «не» (отрицание как неудача).

Prolog применяется в экспертных системах, обработке естественного языка, планировании, проверке правил и прототипировании систем ИИ: правила легко читаются человеком, а механизм вывода встроен в язык.

3.3. Инструменты

Инструмент	**Назначение в работе**
SWI-Prolog 10.0.2	Интерпретатор Prolog: выполнение базы знаний, запросов и тестов
plunit (входит в SWI-Prolog)	Модульные тесты базы знаний
Protege 5.6.9	Редактор онтологий от Стэнфордского университета
HermiT 1.4.3	Reasoner OWL 2 с поддержкой SWRL: классификация, вывод, проверка непротиворечивости
DL Query, SPARQL Query	Вкладки Protege для запросов к онтологии
OntoGraf, OWLViz	Визуализация иерархии классов и связей
SWRL	Язык правил поверх OWL, использован для правила `same_developer/2`

Для связи Prolog и онтологий существуют и библиотеки: модуль `semweb/rdf11` SWI-Prolog позволяет загружать RDF/OWL-файлы и обращаться к триплетам как к фактам Prolog, а `Thea` — работать с аксиомами OWL. В данной работе обе реализации построены независимо, чтобы сравнить подходы в чистом виде.

4. Реализация системы искусственного интеллекта

4.1. База знаний на Prolog

База знаний находится в файле `knowledge_base.pl`. Объекты предметной области — 20 видеоигр, 7 жанров и 6 студий. Свойства игр заданы унарными фактами, связи — бинарными:

```
% game(Game) — Game является видеоигрой.
game(portal_2).
% multiplayer(Game) — игра поддерживает совместную игру.
multiplayer(portal_2).
% has_genre(Game, Genre) — игра Game относится к жанру Genre.
has_genre(portal_2, puzzle).
% developed_by(Game, Studio) — игра Game создана студией Studio.
developed_by(portal_2, valve).
```

Логика принятия решений реализована семью правилами:

Правило	**Смысл**	**Операторы**
`action_game/1`	жанр action или action_rpg	и, или
`social_game/1`	игра с мультиплеером	и
`indie_singleplayer/1`	независимая одиночная игра	и
`narrative_solo/1`	сюжетная игра без мультиплеера	и, не
`game_by_studio/2`	игры заданной студии	и
`same_developer/2`	разные игры одной студии	и, `dif/2`
`recommended_for_beginner/1`	рекомендация новичку	и, или, не

Наиболее показательное правило — рекомендация для начинающего игрока. Оно сочетает все три логические операции:

```
recommended_for_beginner(Game) :-
    game(Game),
    \+ competitive(Game),
    (cozy(Game); has_genre(Game, puzzle)).
```

Цель `game(Game)` стоит первой, чтобы к моменту проверки `\+ competitive(Game)` переменная уже была связана. Отрицание как неудача корректно только для конкретного объекта: для несвязанной переменной `\+ competitive(X)` было бы ложно при наличии хотя бы одной соревновательной игры.

Правило `same_developer/2` находит пары через общую переменную `Studio`, а `dif/2` исключает пару игры с самой собой:

```
same_developer(Game1, Game2) :-
    developed_by(Game1, Studio),
    developed_by(Game2, Studio),
    dif(Game1, Game2).
```

4.2. Онтология в Protege

Онтология хранится в файле `ontology/video_games.ttl` (синтаксис Turtle) и открывается в Protege. Перевод выполнен по правилам:

Prolog	**OWL**
`game/1`	класс `Game` и 20 именованных индивидов
второй аргумент `has_genre/2`, `developed_by/2`	классы `Genre` (7 индивидов), `Studio` (6 индивидов)
`singleplayer/1`, `indie/1`, `story_driven/1`, `cozy/1`	подклассы `Game`; факт — утверждение о типе индивида
`multiplayer/1`, `competitive/1`	подклассы `Game`, закрытые перечислением `owl:oneOf`
`has_genre/2`	объектное свойство `hasGenre`: Game → Genre; обратное `isGenreOf`
`developed_by/2`	функциональное свойство `developedBy`: Game → Studio
`action_game/1`	`ActionGame ≡ Game ⊓ (∃hasGenre.{action} ⊔ ∃hasGenre.{action_rpg})`
`social_game/1`	`SocialGame ≡ Game ⊓ MultiplayerGame`
`indie_singleplayer/1`	`IndieSingleplayerGame ≡ IndieGame ⊓ SingleplayerGame`
`narrative_solo/1`	`NarrativeSoloGame ≡ StoryDrivenGame ⊓ ¬MultiplayerGame`
`game_by_studio/2`	обратное свойство `develops ≡ developedBy⁻`
`same_developer/2`	свойство `sameDeveloperAs` (симметричное, иррефлексивное) и SWRL-правило
`recommended_for_beginner/1`	`BeginnerFriendlyGame ≡ Game ⊓ ¬CompetitiveGame ⊓ (CozyGame ⊔ ∃hasGenre.{puzzle})`

Логические операторы переведены так: «и» — пересечение классов (`owl:intersectionOf`), «или» — объединение (`owl:unionOf`), «не» — дополнение (`owl:complementOf`). Правила, выраженные эквивалентными классами, работают как определения: индивиды не относятся к ним вручную, их принадлежность выводит reasoner.

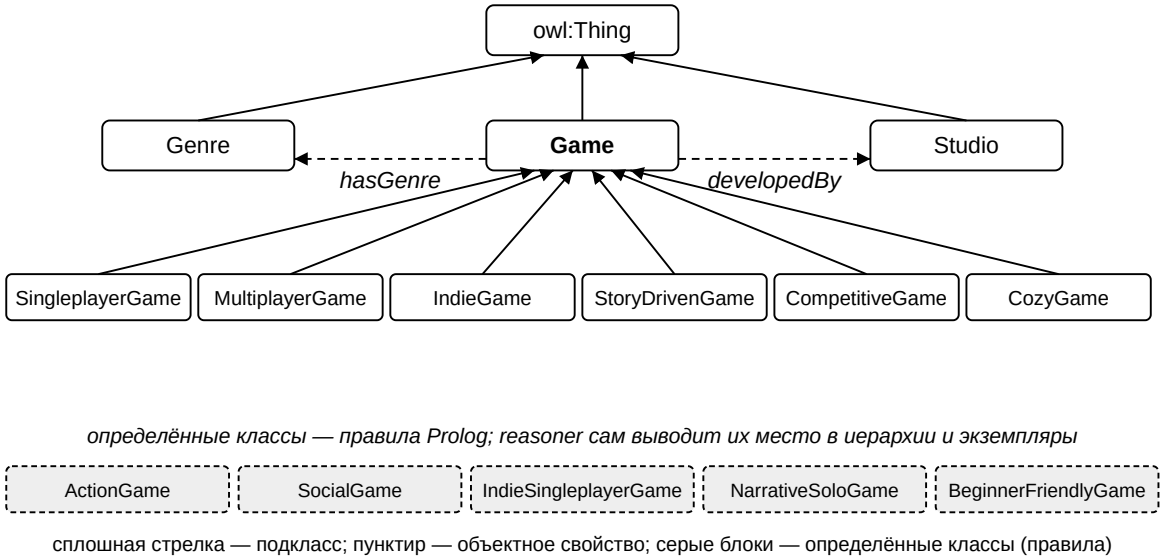


Рисунок 1 — Структура классов и свойств онтологии

Перевод отрицания. Правила `narrative_solo/1` и `recommended_for_beginner/1` используют отрицание как неудачу. Прямой перевод в `¬MultiplayerGame` не даёт результата: из-за предположения об открытом мире reasoner не может заключить, что Hades — не многопользовательская игра, ведь это нигде не сказано. Поэтому в онтологию добавлены две аксиомы, воспроизводящие допущения Prolog:

1. классы `MultiplayerGame` и `CompetitiveGame` объявлены эквивалентными перечислению своих индивидов (`owl:oneOf`) — «других таких игр нет», это аналог замкнутого мира для конкретного предиката;
2. все индивиды объявлены попарно различными (`owl:AllDifferent`) — аналог уникальности имён.

```
:MultiplayerGame owl:equivalentClass [ owl:oneOf ( :portal_2 :dota_2
    :minecraft :counter_strike_2 :terraria :civilization_6 :among_us
    :rocket_league :dark_souls_3 :red_dead_redemption_2 :baldurs_gate_3 ) ] .
```

Перевод `same_developer/2`. Отношение между двумя переменными через третью (общую студию) не выражается классом, поэтому использовано правило SWRL. Атом `differentFrom` играет роль `dif/2`:

```
developedBy(?g1, ?s) ^ developedBy(?g2, ?s) ^ differentFrom(?g1, ?g2)
    -> sameDeveloperAs(?g1, ?g2)
```

Дополнительно заданы аксиомы, которых не было в Prolog: классы `Game`, `Genre`, `Studio` попарно не пересекаются, у свойств указаны домен и диапазон, `developedBy` функционально. Они позволяют reasoner'у обнаруживать ошибки ввода, например, если студию по ошибке указать как жанр игры.

4.3. Тестирование и отладка

Prolog. Файл `tests.pl` содержит 9 модульных тестов plunit: проверку известного и неизвестного факта, бинарного факта и каждого правила. Для правил проверяется полное множество ответов (`set(...)`), а не один ответ, — это ловит как пропущенные, так и лишние решения. Результат запуска:

```
$ swipl -q -s tests.pl -g run_tests,halt
% [1/9] game_knowledge_base:known_game .............. passed
% [2/9] game_knowledge_base:unknown_game ............ passed
% [3/9] game_knowledge_base:binary_fact ............. passed
% [4/9] game_knowledge_base:action_rule ............. passed
% [5/9] game_knowledge_ba..e_singleplayer_rule ...... passed
% [6/9] game_knowledge_ba.._solo_uses_negation ...... passed
% [7/9] game_knowledge_base:valve_games ............. passed
% [8/9] game_knowledge_ba..same_developer_rule ...... passed
% [9/9] game_knowledge_ba..nner_recommendation ...... passed
```

Онтология. Онтология загружена в Protege 5.6.9 и классифицирована reasoner'ом HermiT 1.4.3. Результат: онтология непротиворечива, содержит 252 аксиомы, включая одно SWRL-правило. Выведенные экземпляры определённых классов сопоставлены с ответами Prolog-тестов — совпадение полное (таблица 5.2). SPARQL-запросы из файла `ontology/queries.sparql` также дают те же ответы, что и соответствующие запросы Prolog.

Отладка. При первой проверке классы `NarrativeSoloGame` и `BeginnerFriendlyGame` могут оказаться пустыми — это ожидаемое проявление открытого мира, описанное в п. 4.2. Причина устраняется аксиомами `owl:oneOf` и `owl:AllDifferent`. Также установлено, что результат зависит от reasoner'a: HermiT корректно обрабатывает номиналы (`oneOf`) и SWRL, поэтому он рекомендован для демонстрации.

5. Оценка и интерпретация результатов

5.1. Примеры запросов к базе знаний и онтологии

В таблице 5.1 приведены одни и те же вопросы в трёх формах: запрос Prolog, запрос DL Query (Protege, с выводом reasoner'a) и запрос SPARQL (Protege, по явно заданным фактам).

Таблица 5.1 — Один вопрос в трёх языках запросов

Вопрос	**Prolog**	**DL Query**	**SPARQL**
Portal 2 — игра?	`game(portal_2).`	—	`ASK { :portal_2 a :Game }`
Независимые игры	`indie(G).`	`IndieGame`	`?g a :IndieGame`
Одиночные И сюжетные	`singleplayer(G), story_driven(G).`	`SingleplayerGame and StoryDrivenGame`	два триплета подряд
Спокойные ИЛИ соревновательные	`cozy(G) ; competitive(G).`	`CozyGame or CompetitiveGame`	`UNION`
Без мультиплеера	`game(G), \+ multiplayer(G).`	`Game and not MultiplayerGame`	`FILTER NOT EXISTS`
Игры Valve	`developed_by(G, valve).`	`developedBy value valve`	`?g :developedBy :valve`
Игры той же студии, что Portal 2	`same_developer(G, portal_2).`	`sameDeveloperAs value portal_2`	два триплета + `FILTER(?a != ?b)`
Рекомендации новичку	`recommended_for_beginner(G).`	`BeginnerFriendlyGame`	`FILTER NOT EXISTS` + `UNION`
Студии, выпустившие головоломку	`developed_by(G, S), has_genre(G, puzzle).`	`Studio and develops some (hasGenre value puzzle)`	два триплета

Пример выполнения демонстрационного сценария `demo.pl` (фрагмент):

```
$ swipl -q -s demo.pl
5. Игры БЕЗ мультиплеера:
  [celeste,disco_elysium,hades,half_life_2,hollow_knight,outer_wilds,
   stardew_valley,the_witcher_3,undertale]
8. Сюжетные игры без мультиплеера (правило narrative_solo/1):
  [celeste,disco_elysium,hades,half_life_2,hollow_knight,outer_wilds,
   the_witcher_3,undertale]
10. Рекомендации новичку (правило recommended_for_beginner/1):
  [minecraft,portal_2,stardew_valley]
```

Таблица 5.2 — Сравнение результатов Prolog и онтологии (HermiT)

Правило / класс	**Ответ**	**Prolog**	**OWL**
`action_game` / `ActionGame`	hades, the_witcher_3	✓	✓
`social_game` / `SocialGame`	11 многопользовательских игр	✓	✓
`indie_singleplayer` / `IndieSingleplayerGame`	celeste, disco_elysium, hades, hollow_knight, outer_wilds, stardew_valley, terraria, undertale	✓	✓
`narrative_solo` / `NarrativeSoloGame`	celeste, disco_elysium, hades, half_life_2, hollow_knight, outer_wilds, the_witcher_3, undertale	✓	✓
`game_by_studio(G, valve)` / `valve develops`	dota_2, half_life_2, portal_2	✓	✓
`same_developer` / `sameDeveloperAs`	6 упорядоченных пар игр Valve	✓	✓
`recommended_for_beginner` / `BeginnerFriendlyGame`	minecraft, portal_2, stardew_valley	✓	✓

5.2. Сравнение реализаций

Критерий	**Prolog**	**OWL / Protege**
Модель мира	Замкнутый: недоказуемое ложно	Открытый: неуказанное неизвестно
Уникальность имён	Есть по умолчанию	Нужно заявлять (`AllDifferent`)
Отрицание	Просто: `\+`	Требует закрытия классов (`oneOf`)
Отношения через переменные	Естественно (`same_developer`)	Нужен SWRL
Иерархия и классификация	Описывается вручную правилами	Строится reasoner'ом автоматически
Проверка непротиворечивости	Нет	Есть (disjoint, domain/range, functional)
Обмен знаниями	Свой формат	Стандарт W3C, IRI, импорт других онтологий
Визуализация	Нет	OntoGraf, OWLViz
Произвольные вычисления	Да (полный язык программирования)	Нет (только логический вывод)

5.3. Оценка соответствия требованиям

Все требования задания выполнены: база знаний содержит 73 унарных факта (из них 20 — `game/1`), 15 бинарных фактов и 7 правил; запросы охватывают простой поиск, переменные, «и», «или», «не» и вывод по правилам; код прокомментирован. Онтология содержит 14 классов с иерархией, 5 объектных свойств, 33 индивида и SWRL-правило; она непротиворечива, визуализируется и отвечает на запросы так же, как база знаний. Цель проекта — построить и сравнить два представления знаний — достигнута.

5.4. Интерпретация и возможности развития

Результаты показывают, что одна и та же логика выражается обоими средствами, но с разной ценой. Prolog удобнее для правил, связывающих несколько объектов через переменные, и для отрицания. OWL лучше подходит для описания структуры понятий: reasoner сам выстроил иерархию (например, вывел, что `SocialGame` эквивалентен `MultiplayerGame`, а `NarrativeSoloGame` — подкласс `StoryDrivenGame`) и может проверять корректность данных.

Направления развития:

- расширить базу: указать жанры и студии для всех 20 игр, добавить платформы, год выпуска, рейтинг (свойства-данные OWL);

- добавить профиль игрока (предпочтения, опыт) и правило подбора игр под профиль с ранжированием;
- связать реализации: загрузить онтологию в SWI-Prolog через `semweb/rdf11` и выполнять правила Prolog над её данными;
- импортировать данные из открытых источников (Wikidata) по SPARQL;
- сделать пользовательский интерфейс для ввода предпочтений.

6. Заключение

В ходе работы разработана система поддержки выбора видеоигр в двух представлениях: база знаний на Prolog и онтология OWL 2 в Protege. Обе реализации протестированы и дают совпадающие результаты на всех запросах.

Преимущества разработанной системы:

- знания отделены от механизма вывода: чтобы добавить игру, достаточно дописать факты, правила менять не нужно;
- правила читаются как утверждения на естественном языке, поэтому их может проверить эксперт предметной области;
- ответы выводятся автоматически и объяснимы: всегда можно указать факты и правило, из которых получен результат (в Protege — кнопкой Explanation);
- онтология стандартизирована, непротиворечива и пригодна для совместного использования другими системами.

Потенциальные применения подобных систем: рекомендательные сервисы в игровых магазинах, экспертные системы, каталоги и поиск по смыслу, графы знаний, проверка соответствия правилам (например, возрастным рейтингам), а также образовательные системы.

Главный практический вывод работы — различие предположений о замкнутом и открытом мире. При переводе знаний из Prolog в OWL отрицание и уникальность имён нужно явно описывать аксиомами, иначе reasoner не выведет того, что Prolog считает очевидным. Понимание этого различия необходимо при проектировании любых интеллектуальных систем на основе знаний.

Приложение А. Структура проекта

Файл	Содержание
`knowledge_base.pl`	База знаний Prolog: факты и 7 правил с комментариями
`demo.pl`	Демонстрационный сценарий из 10 запросов
`tests.pl`	9 модульных тестов plunit
`ontology/video_games.ttl`	Онтология OWL 2 для Protege
`ontology/queries.sparql`	10 SPARQL-запросов
`ontology/README.md`	Таблица перевода, DL-запросы, порядок демонстрации

Команды запуска:

```
swipl -q -s demo.pl                          # демонстрация запросов
swipl -q -s tests.pl -g run_tests,halt       # тесты
swipl -s knowledge_base.pl                   # интерактивный режим
protege ontology/video_games.ttl             # онтология в Protege
```